Huffmanov in aritmetični kod

1 Huffmanov kod

Huffmanov kod je optimalen predponski kod za dano porazdelitev verjetnosti simbolov. Ideja je, da vedno združujemo dva najmanj verjetna simbola v novo vozlišče, dokler ne dobimo enega korena.

Postopek je naslednji:

1. Simbole uredimo po naraščajočih verjetnostih.
2. Združimo dva simbola (ali poddrevesi) z najmanjšima verjetnostma.
3. Novi skupni verjetnosti vrnemo v množico in postopek ponavljamo.
4. Na veje drevesa pripišemo bite (npr. levo 0, desno 1).
5. Kodna beseda simbola je zaporedje bitov od korena do lista.

Primer 1

Naj bo

$$A = \{a_1, a_2, a_3, a_4\}, \qquad P = \left\{\tfrac{1}{2}, \tfrac{1}{4}, \tfrac{1}{8}, \tfrac{1}{8}\right\}.$$

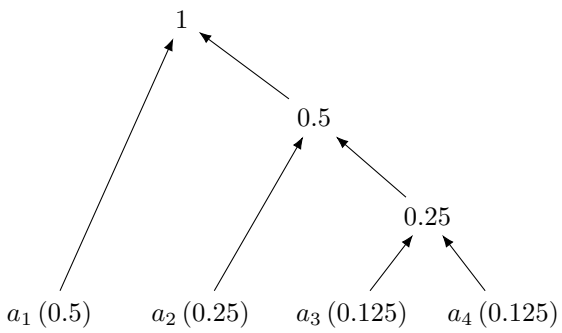
Združujemo:

$$\tfrac{1}{8} + \tfrac{1}{8} = \tfrac{1}{4}, \qquad \tfrac{1}{4} + \tfrac{1}{4} = \tfrac{1}{2}, \qquad \tfrac{1}{2} + \tfrac{1}{2} = 1.$$

Ena možna Huffmanova koda:

Znak	p_i	Kod
a_1	$\frac{1}{2}$	0
a_2	$\frac{1}{4}$	10
a_3	$\frac{1}{8}$	110
a_4	$\frac{1}{8}$	111

Konstrukcijsko drevo (od spodaj navzgor):



Povprečna dolžina:

$$L = \tfrac{1}{2} \cdot 1 + \tfrac{1}{4} \cdot 2 + \tfrac{1}{8} \cdot 3 + \tfrac{1}{8} \cdot 3 = \tfrac{7}{4} = 1.75.$$

Entropija izvora:

$$H = -\sum_i p_i \log_2 p_i = \tfrac{7}{4} = 1.75.$$

Učinkovitost:

$$\eta = \frac{H}{L} = 1 = 100\%.$$

Povzetek:

$$L = 1,75, \quad H = 1,75 \text{ bit/simbol}, \quad \eta = 1,00.$$

Primer 2

Naj bo

$$A = \{a, b, c, d, e\}, \qquad P = \{0.4, 0.2, 0.2, 0.1, 0.1\}.$$

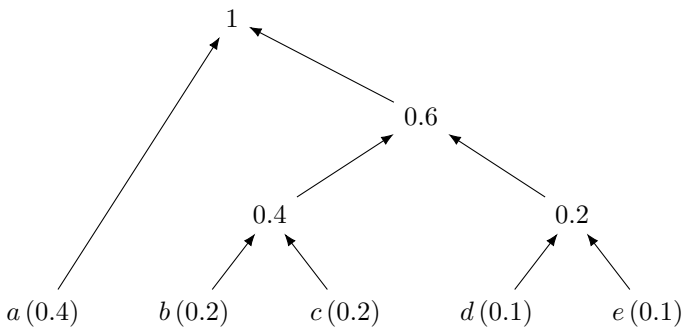
Združovanja (ena možnost):

$$0.1 + 0.1 = 0.2, \quad 0.2 + 0.2 = 0.4, \quad 0.2 + 0.4 = 0.6, \quad 0.4 + 0.6 = 1.$$

Dobimo npr. naslednje kode:

Znak	p_i	Kod
a	0.4	0
b	0.2	100
c	0.2	101
d	0.1	110
e	0.1	111

Konstrukcijsko drevo (od spodaj navzgor):



Povprečna dolžina:

$$L = 0.4 \cdot 1 + 0.2 \cdot 3 + 0.2 \cdot 3 + 0.1 \cdot 3 + 0.1 \cdot 3 = 2.2.$$

Entropija izvora:

$$H \approx 2.122.$$

Učinkovitost:

$$\eta = \frac{H}{L} \approx \frac{2.122}{2.2} \approx 0.965 \approx 96.5\%.$$

Povzetek:

$$L = 2,20, \quad H = 2,12 \text{ bit/simbol}, \quad \eta = 0,96.$$

Primer 3

Naj bo

$$A = \{a, b, c, d, e, f\}, \qquad P = \{0.30, 0.25, 0.20, 0.10, 0.10, 0.05\}.$$

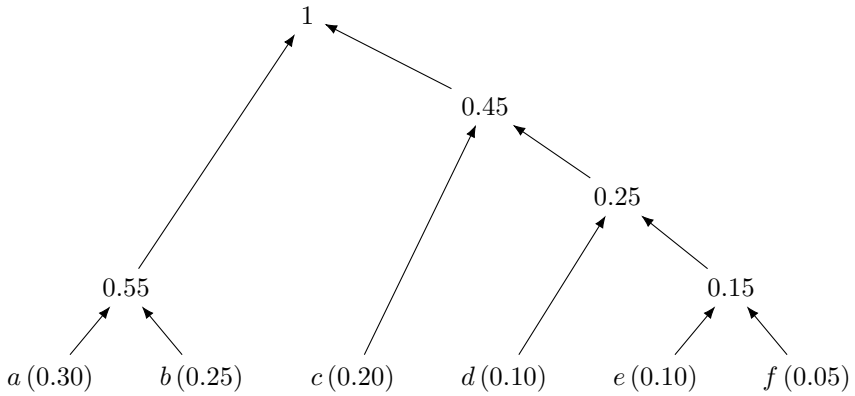
Združovanja:

$$\begin{aligned} 0.05 + 0.10 &= 0.15, \\ 0.10 + 0.15 &= 0.25, \\ 0.20 + 0.25 &= 0.45, \\ 0.25 + 0.30 &= 0.55, \\ 0.45 + 0.55 &= 1. \end{aligned}$$

Ena možna koda:

Znak	p_i	Kod
a	0.30	11
b	0.25	01
c	0.20	00
d	0.10	1010
e	0.10	100
f	0.05	1011

Konstrukcijsko drevo (od spodaj navzgor):



Povprečna dolžina:

$$L = 0.30 \cdot 2 + 0.25 \cdot 2 + 0.20 \cdot 2 + 0.10 \cdot 4 + 0.10 \cdot 3 + 0.05 \cdot 4 = 2.4.$$

Entropija izvora:

$$H \approx 2.366.$$

Učinkovitost:

$$\eta = \frac{H}{L} \approx \frac{2.366}{2.4} \approx 0.986 \approx 98.6\%.$$

Povzetek:

$$L = 2,40, \quad H = 2,37 \text{ bit/simbol}, \quad \eta = 0,99.$$

Primer 4 (spremembe vrstnega reda in prioritetna vrsta)

Naj bo

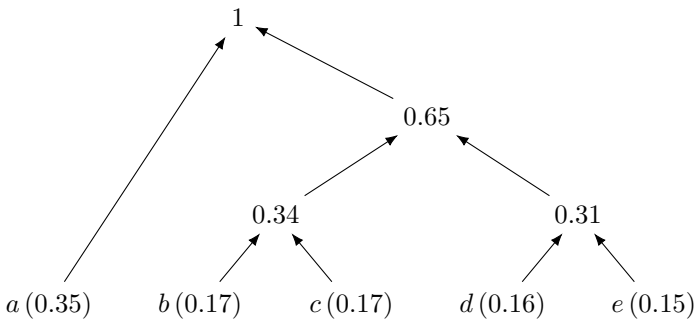
$$A = \{a, b, c, d, e\}, \qquad P = \{0.35, 0.17, 0.17, 0.16, 0.15\}.$$

Ta primer je uporaben zato, ker se vrstni red kandidatov med gradnjo drevesa večkrat spremeni. Če bi po vsakem združevanju ročno ponovno urejali seznam, je postopek nepregleden; s prioritetno vrsto pa vedno vzamemo dva najmanj verjetna elementa v času $O(\log n)$.

Stanja prioritetne vrste (padajoče):

Korak	Elementi v vrsti
Začetek	$a(0.35)$, $c(0.17)$, $b(0.17)$, $d(0.16)$, $e(0.15)$
1	združimo $e + d \to n_1(0.31)$, nova vrsta: $a(0.35)$, $n_1(0.31)$, $c(0.17)$, $b(0.17)$
2	združimo $b + c \to n_2(0.34)$, nova vrsta: $a(0.35)$, $n_2(0.34)$, $n_1(0.31)$
3	združimo $n_1 + n_2 \to n_3(0.65)$, nova vrsta: $n_3(0.65)$, $a(0.35)$
4	združimo $a + n_3 \to 1$ (koren)

Konstrukcijsko drevo (od spodaj navzgor, listi so levo-desno v padajočem vrstnem redu):



Ena možna koda:

Znak	p_i	Kod
a	0.35	0
e	0.15	100
d	0.16	101
b	0.17	110
c	0.17	111

Povprečna dolžina:

$$L = 0.35 \cdot 1 + 0.15 \cdot 3 + 0.16 \cdot 3 + 0.17 \cdot 3 + 0.17 \cdot 3 = 2.30.$$

Entropija izvora:

$$H \approx 2.233.$$

Učinkovitost:

$$\eta = \frac{H}{L} \approx \frac{2.233}{2.30} \approx 0.971 \approx 97.1\%.$$

Povzetek:

$$L = 2,30, \quad H = 2,23 \text{ bit/simbol}, \quad \eta = 0,97.$$

2 Aritmetični kod

Pri aritmetičnem kodiranju celotno sporočilo predstavimo kot eno število iz intervala $[0, 1)$. Simboli ne dobijo fiksnih kodnih besed; namesto tega vsak simbol postopno zoži trenutni interval.

Osnovni postopek:

1. Določimo kumulativne meje simbolov (porazdelitev na $[0, 1)$).
2. Začetni interval je $[L, H) = [0, 1)$.
3. Za vsak simbol s iz sporočila posodobimo interval:

$$\begin{aligned} R &= H - L, \\ H' &= L + R \cdot C_{\text{high}}(s), \\ L' &= L + R \cdot C_{\text{low}}(s). \end{aligned}$$

4. Po zadnjem simbolu izberemo poljubno število iz končnega intervala.

Primer 1

Naj bo abeceda

$$A = \{A, B, C\}, \qquad P = \{0.5, 0.3, 0.2\},$$

in razdelitev intervala:

$$A : [0, 0.5), \quad B : [0.5, 0.8), \quad C : [0.8, 1).$$

Kodiramo sporočilo BAC:

$$\begin{aligned} [0, 1) &\xrightarrow{B} [0.5, 0.8), \\ [0.5, 0.8) &\xrightarrow{A} [0.5, 0.65), \\ [0.5, 0.65) &\xrightarrow{C} [0.62, 0.65). \end{aligned}$$

Veljaven kod je katerokoli število iz tega intervala, npr. $x = 0.63$. Entropija izvora (na simbol) je

$$H \approx 1.485 \text{ bita/simbol}.$$

Za širino končnega intervala $\Delta = 0.65 - 0.62 = 0.03$ in dolžino sporočila $n = 3$ je

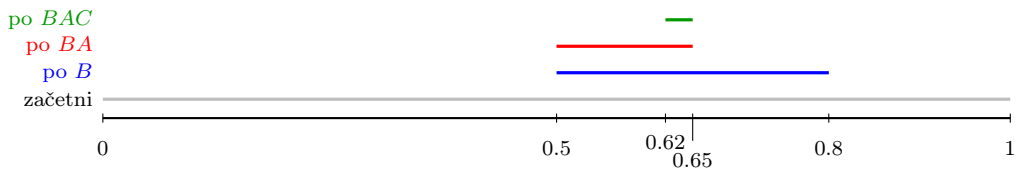
$$R = \frac{-\log_2 \Delta}{n} \approx \frac{-\log_2(0.03)}{3} \approx 1.686 \text{ bita/simbol},$$

zato je učinkovitost

$$\eta = \frac{H}{R} \approx \frac{1.485}{1.686} \approx 0.881 \approx 88.1\%.$$

Povzetek:

$$L = 1.69, \quad H = 1.49 \text{ bit/simbol}, \quad \eta = 0.88.$$



Primer 2

Naj bo abeceda

$$A = \{a, b, c\}, \qquad P = \left\{\tfrac{4}{7}, \tfrac{2}{7}, \tfrac{1}{7}\right\},$$

in meje:

$$a : [0, \tfrac{4}{7}), \quad b : [\tfrac{4}{7}, \tfrac{6}{7}), \quad c : [\tfrac{6}{7}, 1).$$

Kodiramo sporočilo *abca*:

$$\begin{aligned}
[0, 1) &\xrightarrow{a} [0, \tfrac{4}{7}), \\
[0, \tfrac{4}{7}) &\xrightarrow{b} [\tfrac{16}{49}, \tfrac{24}{49}), \\
[\tfrac{16}{49}, \tfrac{24}{49}) &\xrightarrow{c} [\tfrac{160}{343}, \tfrac{168}{343}), \\
[\tfrac{160}{343}, \tfrac{168}{343}) &\xrightarrow{a} [\tfrac{1120}{2401}, \tfrac{1152}{2401}).
\end{aligned}$$

Ena možna kodna vrednost je npr. $\frac{1130}{2401}$. Entropija izvora (na simbol) je

$$H \approx 1.379 \text{ bita/simbol.}$$

Za širino končnega intervala

$$\Delta = \frac{1152 - 1120}{2401} = \frac{32}{2401}$$

in dolžino sporočila $n = 4$ je

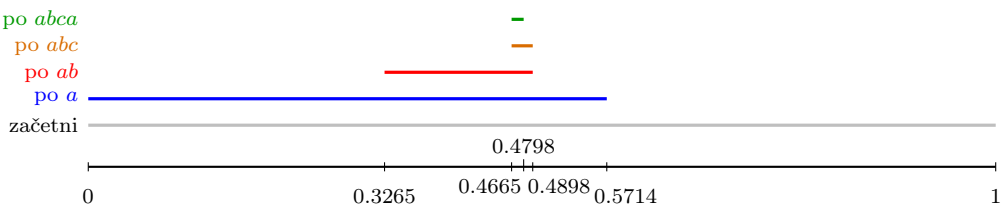
$$R = \frac{-\log_2 \Delta}{n} \approx 1.557 \text{ bita/simbol,}$$

zato je učinkovitost

$$\eta = \frac{H}{R} \approx \frac{1.379}{1.557} \approx 0.886 \approx 88.6\%.$$

Povzetek:

$$L = 1,56, \quad H = 1,38 \text{ bit/simbol}, \quad \eta = 0,89.$$



Primer 3

Naj bo

$$A = \{0, 1\}, \qquad P = \{0.6, 0.4\},$$

in meje

$$0 : [0, 0.6), \qquad 1 : [0.6, 1).$$

Kodiramo sporočilo 101:

$$\begin{aligned}
[0, 1) &\xrightarrow{1} [0.6, 1), \\
[0.6, 1) &\xrightarrow{0} [0.6, 0.84), \\
[0.6, 0.84) &\xrightarrow{1} [0.744, 0.84).
\end{aligned}$$

Veljaven predstavnik je npr. 0.8. Entropija izvora (na simbol) je

$$H \approx 0.971 \text{ bita/simbol.}$$

Za širino končnega intervala $\Delta = 0.84 - 0.744 = 0.096$ in dolžino sporočila $n = 3$ je

$$R = \frac{-\log_2 \Delta}{n} \approx \frac{-\log_2(0.096)}{3} \approx 1.127 \text{ bita/simbol,}$$

zato je učinkovitost

$$\eta = \frac{H}{R} \approx \frac{0.971}{1.127} \approx 0.862 \approx 86.2\%.$$

Povzetek:

$$L = 1.13, \quad H = 0.97 \text{ bit/simbol}, \quad \eta = 0.86.$$

Aritmetični kod brez floating point/realnih števil

V praksi se aritmetični kod skoraj vedno izvaja celoštevilsko (range coding), brez uporabe tipa `float`/`double`. Namesto intervala v realnih številih uporabimo celoštevilski interval

$$[\texttt{low}, \texttt{high}] \subseteq [0, 2^n - 1].$$

Naj bo `total` vsota frekvenc, `cumLow[s]` in `cumHigh[s]` pa kumulativni meji simbola s v diskretni porazdelitvi. Posodobitev za simbol s je:

$$\begin{aligned}
\texttt{range} &= \texttt{high} - \texttt{low} + 1, \\
\texttt{high} &= \texttt{low} + \left\lfloor \frac{\texttt{range} \cdot \texttt{cumHigh[s]}}{\texttt{total}} \right\rfloor - 1, \\
\texttt{low} &= \texttt{low} + \left\lfloor \frac{\texttt{range} \cdot \texttt{cumLow[s]}}{\texttt{total}} \right\rfloor.
\end{aligned}$$

Nato izvajamo renormalizacijo (E1/E2/E3 pravila):

- če sta `low` in `high` v spodnji polovici, izpišemo bit 0;
- če sta v zgornji polovici, izpišemo bit 1;
- če sta v srednjih dveh četrtinah (underflow primer), povečamo števec odloženih bitov.

Po vsakem od teh primerov interval skaliramo (premik v levo za 1 bit), da ohranimo numerično stabilnost.

Kratek integer primer. Uporabimo $n = 16$, torej `low=0`, `high=65535`. Naj bodo frekvence

$$a : 5, \quad b : 3, \quad c : 2, \quad \texttt{total} = 10,$$

za sporočilo ba:

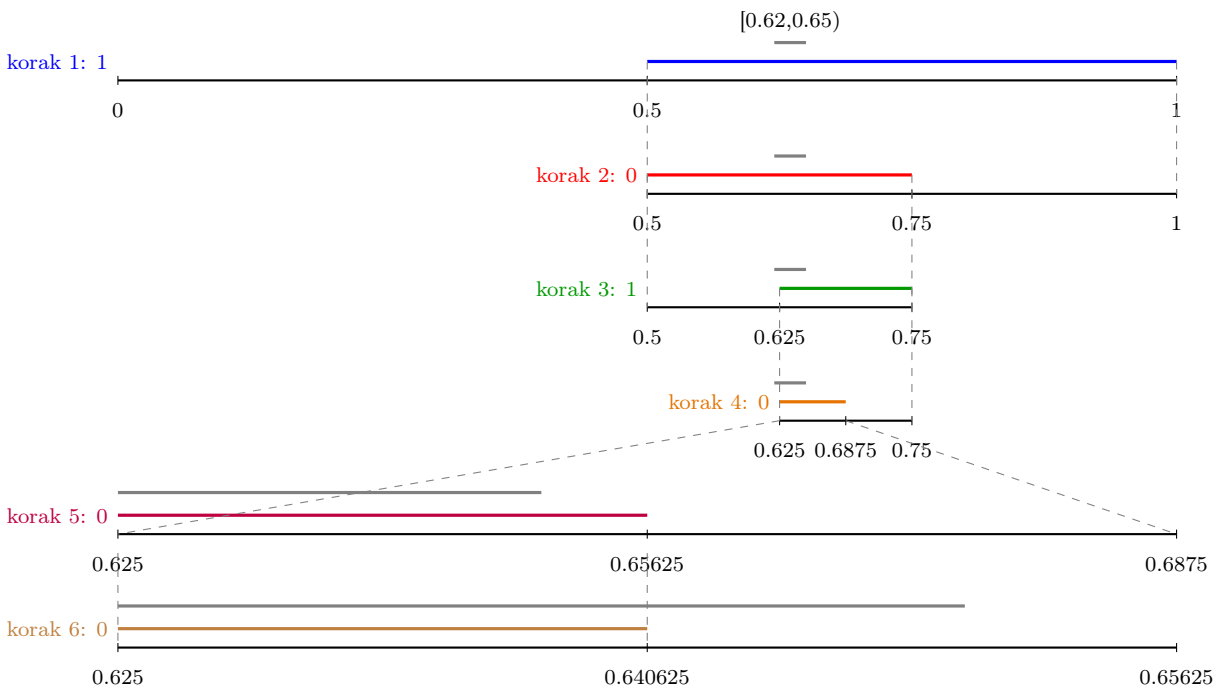
$$\begin{aligned}
\text{po } b: &\quad [\texttt{low}, \texttt{high}] = [32768, 52427], \\
\text{po } a: &\quad [32768, 42597].
\end{aligned}$$

V nobenem koraku nismo uporabili floating point, samo celoštevilsko aritmetiko in bitne premike.

Kodiranje aritmetičnega intervala v binarno število

Aritmetični kod predstavlja sporočilo kot interval [L,H) v [0,1). Da dobimo binarno predstavitev, začnemo z [0,1) in vedno delimo na dva enaka dela. Izberemo tisti del, ki vsebuje interval [L,H), če obstaja, sicer tistega, ki vsebuje večji del intervala [L,H). Dodelimo 0 za spodnji del, 1 za zgornji. Ponavljamo, dokler trenutni binarno interval ni popolnoma vsebovan v [L,H).

Primer 1: Interval [0.62,0.65).

1. Začetek: [0,1), [0.62,0.65) $\subseteq$ [0.5,1), izberemo [0.5,1), bit 1.
2. Sedaj [0.5,1), delimo na [0.5,0.75), [0.75,1). [0.62,0.65) $\subseteq$ [0.5,0.75), izberemo [0.5,0.75), bit 0.
3. Sedaj [0.5,0.75), delimo na [0.5,0.625), [0.625,0.75). Nobena ne vsebuje celega [0.62,0.65), ampak [0.625,0.75) vsebuje večji del (dolžina $0.025 > 0.005$), izberemo [0.625,0.75), bit 1.
4. Sedaj [0.625,0.75), delimo na [0.625,0.6875), [0.6875,0.75). [0.62,0.65) $\subseteq$ [0.625,0.6875), izberemo [0.625,0.6875), bit 0.
5. Sedaj [0.625,0.6875), delimo na [0.625,0.65625), [0.65625,0.6875). [0.62,0.65) $\subseteq$ [0.625,0.65625), izberemo [0.625,0.65625), bit 0.
6. Sedaj [0.625,0.65625), delimo na [0.625,0.640625), [0.640625,0.65625). Nobena ne vsebuje celega, [0.625,0.640625) vsebuje večji del ($0.015 > 0.01$), izberemo [0.625,0.640625), bit 0.

Binarno število 0.101000... = 0.625, ki je v [0.62,0.65).